

Building your project page

Copy the folder, edit three files, add your figures, and publish. No web development experience assumed.

No web development experience is assumed. You will edit three text files, drop your figures into a folder, and flip one setting on GitHub. Budget about an hour for the first one.

Contents

1. [What you are getting](#)
2. [Before you start](#)
3. [Copy the folder into your repository](#)
4. [Put it online straight away](#)
5. [Set the two address settings](#)
6. [Title, venue and tagline](#)
7. [Authors and affiliations](#)
8. [The buttons](#)
9. [Your figures](#)
10. [The teaser](#)
11. [Writing the page body](#)
12. [Tables](#)
13. [Highlight cards and headline numbers](#)
14. [Equations](#)
15. [The citation block](#)
16. [Link previews](#)
17. [Final checklist](#)
18. [Troubleshooting](#)
19. [Optional: preview on your own machine](#)
20. [Optional: changing colours and fonts](#)

1. What you are getting

A single scrolling page with a sticky navigation bar, containing: a title block with your authors and links, a teaser figure, and then whatever sections you write — abstract, method, results, ablations, citation.

Included without any work from you:

- Light and dark themes that follow the visitor's system setting, with a toggle
- Figures that enlarge when clicked

- Tables that scroll sideways on phones instead of breaking the layout
- Equations via KaTeX
- A copy-to-clipboard button on the citation
- Previews when the link is shared on X, LinkedIn or Slack
- Structured data so Google indexes it as a scholarly article

You will edit exactly three files:

File	Holds
<code>_config.yml</code>	Title, venue, tagline, buttons, navigation, teaser, BibTeX
<code>_data/authors.yml</code>	Authors, affiliations, photos, institution logos
<code>index.md</code>	The body of the page

Everything in `_layouts/`, `_includes/` and `assets/css/` is machinery. You can ignore it.

2. Before you start

You need:

- **A GitHub account** and a repository for your paper. It can be an existing code repository — the page lives in a subfolder and will not get in the way.
- **Your figures**, as image files. See [step 9](#) for formats.
- **A text editor**. Any will do. GitHub's own web editor is enough.

You do **not** need to install Ruby, Jekyll, Node, or anything else. GitHub builds the site for you.

A warning about YAML. Two of the three files you edit are YAML, where **indentation is meaningful** and must be **spaces, never tabs**. If the page stops building after an edit, an indentation slip is the first thing to check. Copy the shape of the surrounding lines and you will be fine.

3. Copy the folder into your repository

Copy this folder into your repository and rename it to `docs` :

```
cp -R docs_template /path/to/your-repo/docs
```

Renaming it is safe. Nothing inside the folder refers to its own name. The name `docs` matters only because GitHub Pages offers it as a ready-made option; the alternative is your repository root, which is messier.

Your repository should now look like:

```
your-repo/
├── docs/           ← the website
│   ├── _config.yml
│   ├── _data/authors.yml
│   ├── index.md
│   └── assets/
└── ...           ← your code, unchanged
```

Commit and push.

4. Put it online straight away

Do this now, before writing any content. It takes two minutes and means that when something breaks later you will know which change broke it.

1. Go to your repository on GitHub
2. **Settings** → **Pages** (left sidebar)
3. Under **Build and deployment**, set **Source** to **Deploy from a branch**
4. Set the branch to `main` and the folder to `/docs`
5. Click **Save**

Wait a minute or two, then reload the Settings → Pages screen. It will show your address, something like `https://yourname.github.io/your-repo/`. Open it.

You will see the template with its placeholder text, and it will probably look unstyled — plain black text on white with no layout. That is expected. Fix it in the next step.

5. Set the two address settings

This is the single most common thing to get wrong, so it gets its own step.

Open `_config.yml` and find:

```
url: "https://your-github-username.github.io"
baseurl: "/your-repo-name"
```

`url` is the origin, `baseurl` is the path. Match them to the address GitHub gave you in step 4:

Your site is at	url	baseurl
<code>https://jane.github.io/my-paper/</code>	<code>https://jane.github.io</code>	<code>/my-paper</code>
<code>https://jane.github.io/</code>	<code>https://jane.github.io</code>	<code>""</code>
<code>https://mypaper.com/</code>	<code>https://mypaper.com</code>	<code>""</code>

The middle row is for a repository named exactly `jane.github.io`. The bottom row is for a custom domain.

`baseurl` is your repository name, not your folder name. It takes a leading slash and no trailing slash. Getting it wrong is what makes the page appear unstyled: the browser looks for the stylesheet at the wrong address.

Commit, push, wait a minute, reload. The page should now be laid out properly.

6. Title, venue and tagline

Still in `_config.yml`, under `project:`:

```
project:
  venue: "CVPR 2026"
  title: "Your Short Title"
  tagline: "One sentence saying what the paper does and why it matters."
```

- **venue** is the small coloured line above the title. Use the conference, or "Preprint", or "" to hide it.
- **title** is the big headline. Keep it short — three or four words. It is also used in the navigation bar, where a long title gets truncated.
- **tagline** is the sentence underneath. This is the elevator pitch, not the abstract. The abstract goes in `index.md`.

Separately, near the top of the file, set the two fields search engines use:

```
title: "Your Full Paper Title: Including The Subtitle"
description: "One or two sentences. Shown in Google results and link previews."
```

Note that there are two `title` fields and they are different: the one at the top of the file is the full paper title for browser tabs and search results, and the one under `project:` is the short headline shown on the page.

7. Authors and affiliations

Open `_data/authors.yml`. Affiliations are defined once, then referenced by key:

```
affiliations:
  mit:
    name: "MIT CSAIL"
    url: "https://csail.mit.edu"
  google:
    name: "Google DeepMind"
    url: "https://deepmind.google"
```

Then each author lists the keys that apply to them:

```
list:
  - name: "Jane Doe"
    url: "https://janedoe.com"      # personal page; omit or "" for no link
    photo: "jane-doe.jpg"          # in assets/images/authors/ ; "" for none
    affil: [mit]
    equal: true                     # adds * for equal contribution

  - name: "John Smith"
    url: ""
    photo: ""
    affil: [mit, google]           # two institutions, both shown
```

Two author layouts

The template ships with the **compact conference-style line**: names side by side with numbered affiliation superscripts, affiliations listed underneath. It suits long author lists and takes very little vertical space.

```
show_photos: false      # the default
```

To use **photos** instead — a row of circles, each with a name and affiliation:

```
show_photos: true
```

Photos are still optional in that mode. Any author without one gets a circle with their initials, which looks deliberate rather than unfinished, so you can publish today and add photos as they arrive. Put square images in `assets/images/authors/` and give just the filename, not a path. They display at 44 px inside a circle, so 400×400 is ample; keep each under about 100 KB.

Either way, set `equal: true` on the relevant authors to get `*` markers and an “Equal contribution” note underneath.

Institution logos (optional)

A row of logos under the affiliations. Put files in `assets/images/logos/`, then:

```
logos:
  - { src: "logos/mit.svg", alt: "MIT", url: "https://mit.edu" }
  - { src: "logos/google.svg", alt: "Google DeepMind" }
```

Note the `logos/` prefix here — logo paths are relative to `assets/images/`, whereas author photos are relative to `assets/images/authors/`. Leave `logos: []` to hide the row.

Logos are inverted in dark mode, which is right for the usual dark-on-transparent logo. If yours comes out wrong, supply a light-on-transparent version and delete the invert rule in `assets/css/main.css` — search for `.hero__logos`.

8. The buttons

Back in `_config.yml`:

```
links:
  - label: "Paper"
    url: "https://arxiv.org/pdf/0000.00000"
    icon: "paper"
    primary: true
  - label: "Code"
    url: "https://github.com/you/your-repo"
    icon: "github"
  - label: "BibTeX"
    url: "#bibtex"
    icon: "quote"
```

Available icons: `paper`, `arxiv`, `github`, `video`, `data`, `poster`, `slides`, `quote`, `link`.

- Mark **one** entry `primary: true` to give it the filled blue style. That one also appears as a small button in the navigation bar on wide screens.
- `url: "#bibtex"` scrolls to the citation block rather than leaving the page.
- **Delete entries you do not have.** A button pointing at `"#"` still renders, and a reader who clicks it and gets nothing is worse served than one who never saw the button.

9. Your figures

Put them in `assets/figures/`. Delete the placeholder files that ship with the template once yours are in.

Which format

Your figure is	Use	Why
A diagram or plot from LaTeX/Illustrator (PDF)	SVG	Stays sharp at any zoom, usually smaller than PNG
A plot from matplotlib	PNG	Crisp lines; or save as SVG directly
Photographs, qualitative result grids	JPEG	A quarter the size of PNG at the same visible quality
An animation	MP4	Far smaller than GIF; the template loops it silently

Convert a PDF figure to SVG with one command (install poppler first — `brew install poppler` or `apt install poppler-utils`):

```
pdftocairo -svg figure.pdf assets/figures/figure.svg
```

Size them before committing

Resize anything wider than about 2000 px. The page never displays a figure larger than that, so the extra pixels only slow down loading. A 5 MB PNG screenshot is typically a 400 KB JPEG with no visible difference. On macOS:

```
sips -Z 2000 -s format jpeg -s formatOptions 90 big.png --out assets/figures/small.jpg
```

Aim for the whole `assets/figures/` folder to come in under about 5 MB.

Putting a figure on the page

In `index.md`:

```
{% include figure.html
  src="figures/method.png"
  alt="What the figure shows, described for screen readers."
  caption="**Method overview.** The takeaway first, then the parts."
  width="page" %}
```

Parameter	Meaning
<code>src</code>	Path inside <code>assets/</code> , so <code>figures/x.png</code> for <code>assets/figures/x.png</code>
<code>alt</code>	Description for screen readers and for when the image fails to load
<code>caption</code>	Optional. Markdown works; bold the takeaway
<code>width</code>	<code>page</code> (default) lines the figure up exactly with the tables and body text · <code>text</code> is narrower, for small diagrams · <code>wide</code> reclaims the heading column
<code>pad</code>	<code>"true"</code> adds space inside the frame — use for vector diagrams and anything with a transparent background
<code>plain</code>	<code>"true"</code> removes the frame — use for photos that already fill it
<code>zoom</code>	<code>"false"</code> disables click-to-enlarge

Use `width="wide"` sparingly — once or twice per page, for a figure that genuinely needs the room, such as a multi-panel comparison grid. It is an emphasis device and stops working if everything uses it.

Point an `.mp4` at `src` and it is embedded as a muted, looping, autoplaying video instead of an image. Everything else works the same.

10. The teaser

The figure directly under the title. In `_config.yml`:

```
teaser:
  src: "figures/teaser.jpg"
  alt: "Description for screen readers."
  caption: "**The takeaway in bold.** Then a sentence of explanation."
```

Prefer something **wide**, roughly 2:1 or wider, so more of it is visible when the page opens.

A tall or near-square teaser still works: its height is capped at 76% of the viewport so it can never swallow the whole first screen, and it is centred with click-to-enlarge for the full-size version. If your teaser is wide and short and you would rather it filled the content width, raise or delete the `max-height` in the `TEASER` block of `assets/css/main.css`.

Set `src: ""` to remove the teaser entirely.

11. Writing the page body

`index.md` is ordinary markdown. `##` makes a section heading, `###` a subsection, `bold`, `italic`, `[text](url)` for links.

The one addition: **give each main section an id** with `{#...}` so the navigation bar can link to it.

```
## Abstract {#overview}

Your abstract, as one or two paragraphs.

## Method {#method}

What you did.

### A subsection

Detail.
```

Then list those ids in `_config.yml`:

```
nav:
  - { label: "Overview", id: "overview" }
  - { label: "Method", id: "method" }
  - { label: "Results", id: "results" }
  - { label: "BibTeX", id: "bibtex" }
```

The `label` is what appears in the bar; the `id` must match the `{#...}` exactly. Keep it to **five items or fewer** so it stays on one line. `bibtex` is built in — the citation section always has that id.

Sections without a nav entry are fine, and the bar highlights whichever section you are currently reading.

The bar is hidden when the page first loads, so visitors open on your title and teaser rather than on navigation, and it slides in once they scroll past 120 px. Two ways to change that:

- **Different trigger point** — edit `REVEAL_AT` in `assets/js/main.js`.
- **Always visible** — delete `is-hidden` from the `<header>` tag in `_includes/topbar.html`, and change `position: fixed` to `position: sticky` in the `TOPBAR` block of `assets/css/main.css` so it reserves its own space instead of overlapping your title.

12. Tables

Write plain markdown tables. Right-align numeric columns with `---` so the digits line up, and bold the best result:

```
| Method | Venue | mAP ↓ |
|:-----|:-----|-----:|
| Baseline | CVPR 2024 | 27.2 |
| **Ours** | — | **0.97** |
```

Add a caption on the line straight after the table, ending with `{: .caption}`:

```
**Table 1.** What the table shows.  
{: .caption}
```

Long tables scroll sideways on phones on their own — no action needed.

For a table markdown cannot express — spanning headers, multi-level columns — write ordinary HTML

`<table>` markup instead. It picks up identical styling. In that case you can also add `class="t-ours"` to a `<tr>` to tint your own method's row, and `class="t-sep"` to draw a group separator above a row.

13. Highlight cards and headline numbers

Two components for the top of the page. Both are defined in the **front matter** — the block between `---` markers at the very top of `index.md` — and then placed in the body.

```
---  
highlights:  
  - title: "First point"  
    text: "A sentence or two."  
  - title: "Second point"  
    text: "Three or four cards works best."  
  
stats:  
  - value: "0.97"  
    label: "mAP on COCO"  
    note: "down from 42.1"  
  - value: "4x"  
    label: "faster than prior work"  
---
```

Then wherever you want them:

```
{% include highlights.html %}  
{% include stats.html %}
```

Delete the `highlights:` or `stats:` block and the corresponding `include` line if you do not want them.

14. Equations

Set `math: true` near the top of `_config.yml` (or `false` if your page has no equations — it saves about 300 KB of downloads).

Use `$$ ... $$` for all equations, inline and display alike.

The perturbation δ is bounded by ϵ .

```

$$\delta^{\star} = \arg\min_{\|\delta\|_{\infty} \leq \epsilon} \mathcal{L}(x + \delta)$$

```

Put `$$` on lines of its own for a centred display equation; keep it inline for an inline one.

Do not use single `$ ... $`. This is the one genuine trap in the template. Markdown runs before the equation renderer, and it reads the two underscores in `$a_\theta \circ b_\theta$` as an *emphasis* marker — half your equation silently turns into italic text. It also eats the backslash in `\{` and `\}`, so set braces vanish. Doubling the dollars hands the text to the maths renderer untouched and avoids all of it.

The exception is figure captions. Those are processed differently, so inside a `caption="..."` use single `$ ... $` and double every backslash:

```
caption="A perturbation bounded by  $\epsilon$  in  $\ell_\infty$ ."
```

15. The citation block

In `_config.yml`. The `|` matters — it preserves the line breaks:

```
bibtex: |
  @inproceedings{doe2026title,
    title      = {Your Paper Title},
    author     = {Doe, Jane and Smith, John},
    booktitle  = {CVPR},
    year       = {2026}
  }
```

A copy button is added automatically. Set `bibtex: ""` to hide the section.

Just above it you can add an optional paragraph for funding or thanks:

```
acknowledgements: "This work was supported by ..."
```

16. Link previews

When someone shares your page, the image and text shown come from:

```
description: "One or two sentences."      # top of _config.yml
social_preview: "figures/social-preview.jpg"
```

Make a **1200×630** image — usually your teaser figure cropped — and put it in `assets/figures/`. Note the path is relative to `assets/`, like figure paths.

To check it, paste your URL into any Slack channel or private message. Social platforms cache aggressively, so test before you announce, not after.

17. Final checklist

Before sharing the link:

- ☐ `url` and `baseurl` are correct and the page is styled

- ☐ Every button goes somewhere real; placeholders removed
- ☐ Author names spelled correctly, affiliations right, links work
- ☐ Every figure has a meaningful `alt`
- ☐ Placeholder figures deleted from `assets/figures/`
- ☐ `assets/figures/` is under about 5 MB
- ☐ Every nav item scrolls to its section
- ☐ Equations render as maths, with no stray italics
- ☐ BibTeX is correct and the copy button works
- ☐ Opened on a phone
- ☐ Toggled dark mode
- ☐ Link preview checked by pasting into Slack

18. Troubleshooting

Symptom	Cause and fix
Page has no styling — plain text on white	<code>baseurl</code> is wrong. It must be <code>/your-repo-name</code> : leading slash, no trailing slash. See step 5
Images are broken but text is fine	Check the path is relative to <code>assets/</code> (<code>figures/x.png</code> , not <code>assets/figures/x.png</code> or <code>/figures/x.png</code>), and that the filename case matches exactly — GitHub's servers are case-sensitive even though macOS and Windows are not
Changes are not showing up	Builds take a minute. Check the Actions tab for a failed build; a YAML indentation error will stop it. Then hard-refresh (<code>Cmd/Ctrl + Shift + R</code>)
Half an equation is in italics	You used single <code>\$...\$</code> . Switch to <code>\$\$...\$\$</code> — see step 14
Equations show as raw LaTeX	<code>math: true</code> is not set in <code>_config.yml</code>
A nav item does nothing	The <code>id</code> in <code>_config.yml</code> does not match the <code>{#...}</code> on the heading. They are case-sensitive
Author shows initials instead of a photo	The <code>photo:</code> filename does not match the file in <code>assets/images/authors/</code> . Give just the filename, no path
Build fails after editing YAML	Indentation. Use spaces, never tabs, and copy the shape of the surrounding lines
Page loads slowly	An oversized figure. See step 9

19. Optional: preview on your own machine

Not required — pushing to GitHub is a perfectly good preview loop, and skipping this saves you a Ruby installation. Worth it only if you are iterating heavily.

```
cd docs
bundle install
bundle exec jekyll serve --livereload
```

Then open the address it prints, typically `http://127.0.0.1:4000/your-repo-name/`.

Edits to `index.md` reload automatically. **Edits to `_config.yml` need a restart** — stop with `Ctrl-C` and run the command again. This catches people out: if a config change seems to do nothing, restart before assuming it is broken.

20. Optional: changing colours and fonts

Everything visual is defined as variables at the top of `assets/css/main.css`, in a block marked `TOKENS`.

To change the accent colour, edit `--accent` — but note it appears in **three** places: the light theme (`:root`), the dark theme media query, and the explicit dark theme (`:root[data-theme="dark"]`). Change all three, using a lighter shade for the dark ones so it stays readable.

```
--accent: #0b57d0;          /* links, the venue line, the active nav item */
--accent-hover: #1a73e8;
--accent-soft: #e8f0fe;     /* tinted backgrounds */
```

The filled button is deliberately *not* the accent colour — it is near-black in light mode and near-white in dark mode, so the call to action reads as neutral while links stay blue. It has its own tokens, which also appear in all three theme blocks:

```
--btn-bg: #1f1f1f;          /* the filled "Paper" button */
--btn-bg-hover: #3c4043;
--btn-ink: #ffffff;          /* its label */
```

To change the fonts, edit `--font-display` (headings) and `--font-text` (body), and update the Google Fonts `<link>` in `_includes/head.html` to load them.

Spacing, corner radius and the page width are all in the same block:

```
--page: 1140px;             /* content width */
--measure: 760px;           /* text column width – keep this near 60–75 characters */
```

A note on the design

On screens wider than 1024px the body splits into two columns: **section headings hang in a narrow left column**, and everything else — text, tables, figures, cards — shares one content column with a single left and

right edge. Below 1024px it collapses back to one column with the heading above its section.

That shared edge is doing most of the work of making the page look considered. If you add components of your own, put them in the content column too rather than inventing a third width.

You do not need to mark up the two columns yourself; it falls out of how the page is styled. Just write `##` `Heading` followed by your content as normal.

The other deliberate choice is that figures keep a **light background even in dark mode**. Scientific figures are almost always black-on-transparent, and a dark background makes their axes and labels disappear. If you have a figure designed for a dark background, pass `plain="true"` to drop the frame.

Found something wrong or confusing in this guide? It is `GUIDE.md` in this folder — corrections welcome.